HS3

Carsten Burgard Tomas Dado Jonas Eschle
Matthew Feickert Cornelius Grunwald Alexander Held
Jerry Ling Robin Pelkner Jonas Rembser
Oliver Schulz

2025-09-01

About this document

To the extent possible under law, the authors have waived all copyright and related or neighboring rights to this document. For details, please refer to the *Creative Commons* `CC0` license [1].
This work is published from: CERN, Geneva.

1 Introduction

This section is non-normative.

It is widely agreed upon that the publication of likelihood models from high energy physics experiments is imperative for the preservation and public access to these results. Detailed arguments have been made elsewhere already [2]. This document sets out to provide a standardized format to publish and archive statistical models of any size and across a wide range of mathematical formalisms in a way that is readable by both humans and machines. With the introduction of `pyhf` [3, 4], a `JSON` format for likelihood serialization has been put forward. However, an interoperable format that encompasses likelihoods with a scope beyond stacks of binned histograms was lacking. With the release of `ROOT` 6.26/00 [5] and the experimental `RooJSONFactoryWSTool` therein, this gap has now been filled. This document sets out to document the syntax and features of the Statistics Serialization Standard (HS^3) for likelihoods and statistical models in general, as to be adopted by any HS^3-compatible statistics framework. The examples in this document employ the `JSON` notation, but are intended to encompass also representations as `YAML` or `TOML`.

1.1 How to use this document

Developers of statistical toolkits are invited to specifically refer to versions of this document relating to the support of this standard in their respective implementations. Please note that this document as well as the HS^3 standard are still in development and can still undergo minor and major changes in the future. This document describes the syntax of HS^3 v0.2.9.

1.2 Statistical semantics of HS^3

1.2.1 Statistical models, probability distributions and parameters

HS^3 takes a "forward-modelling" approach throughout: a statistical model m maps a space Θ of free parameters θ to a space of probability distributions that describe the possible outcomes of a specific experiment. For any given parameters θ, the model returns a concrete probability distribution $m(\theta)$. Observed data x is then treated as random variate, presumed to be drawn from the model: $x \sim m(\theta)$. Parameters in HS^3 are always named, so semantically the elements of a parameter space Θ are records θ (i.e. tuples in which every entry has a name). Even if there is only a single free parameter, θ should be thought of as a single-entry record. In the current version of this standard, parameter tuples must be flat and only consist of real numbers (or strings for categorical parameters); vector-valued or nested entries are not supported yet. Future versions of the standard will likely be less restrictive, especially with respect to discrete or vector-valued parameters. Parameter domains defined as part of this standard may be of various types, even though the current version only supports product domains. Future versions are likely to be less restrictive in this regard. Mathematically and computationally, it is often convenient to treat parameters as flat real-valued vectors instead of records, it is the responsibility of the implementation to map between these different views of parameters when and where necessary. Probability distributions in HS^3 (see Distributions) are typically parameterized. Any instance of a distribution that has some of its parameters bound to names instead of concrete values, e.g. $m = d_{\mu=a,\sigma=0.7,\lambda=b,\ldots}$, constitutes a valid statistical model $m(\theta)$ with model parameters $\theta = (a, b)$. When probability distributions are combined via a Cartesian product (see Product distribution), then the records that contain their free parameter values are concatenated. So $m = d_{\mu=a,\sigma=b} \times d_{\mu=c,\sigma=0.7}$ constitutes a model $m(\theta)$ with model parameters $\theta = (a, b, c)$. Distribution parameters may also be bound to the output of functions, and if those functions have inputs that are bound to names instead of values (or again bound to such functions, recursively), then those names become part of the model parameters. A configuration $m = d_{\mu=a,\sigma=0.7,\lambda=f}, f = \texttt{sum}(4.2, g), g = \texttt{sum}(1.3, b)$, for example, also constitutes a (different) model $m(\theta)$ with parameters $\theta = (a, b, c)$. If all parameter values of a probability distribution are set to concrete values, so if there are not directly (or indirectly via functions) bound to names, we call this probability distribution a concrete distribution here. Such distributions can be used

as Bayesian priors (see Bayesian inference).

The variates of all distributions (so the possible results of random draws from them) are currently always records (this may be relaxed in future versions of this standard). So even instances of univariate distributions, like the normal distribution, have variates structured like $(a = \dots,)$. Individual names are assigned to each entry of such record-valued variates.

Future versions of the standard may support array-shaped and record-of-arrays-shaped variates and parameters.

1.2.2 Global namespace

The names of all named HS^3 objects (distributions, functions, likelihoods, data, parameters, variables, domains, parameter points, etc.) form a single namespace and must be unique within an HS^3 document. The names assigned to variate record entries and data fields/axes are not part of the global namespace.

HS^3 implementations will typically provide a renaming/aliasing mechanism that can be used to resolve name clashes when using multiple HS^3 documents together, e.g. for combined analyses.

1.2.3 Probability density functions (PDFs)

Statistics literature often discriminates between probability density functions (PDF) for continuous probability distributions and probability mass functions (PMF) for discrete probability distributions. This standard use the term PDF for both continuous and discrete distributions. The concept of density is to be understood in terms of densities in the realm of measure theory here, that is the density of a probability measure (distribution) is its Radon-Nikodym derivative in respect to an (implied) reference measure. The choice of reference measure would be arbitrary in principle, which scales likelihood functions (Sec. Likelihood) by a constant factor that depends on choice of reference. In this standard, a specific reference measures is implied for each probability distribution, typically the Lebesgue measure for continuous distributions and the counting measure for discrete distributions. The standard aims to match the PDF (resp. PMF) most commonly used in literature for each specific probability distribution and the mathematical form of the PDF is documented explicitly for each distribution in the standard. So within HS^3, probability densities and likelihood functions are unambiguous. Here we use $\mathrm{PDF}(m(\theta), x)$ to denote the density value of the probability distribution/measure m, parameterized by θ, at the point/variate x, in respect to the implied reference for m.

1.2.4 Observed and simulated data

The term data refers here to any collection of values that represents the outcome of an experiment. Data takes the same form as variates of distributions (see Distributions, i. e. a record of real values or vectors. To compare given data with

a given model, the names and shapes of the data entries must match the variates of the probability distributions $m(\theta)$ that the model returns for a specific choice of parameters θ. This means that given a model m and concrete parameter values θ, drawing a set of random values from the probability distribution $m(\theta)$ produces a valid set of simulated observations. Implementations can use this to provide mock-data generation capabilities.

1.2.5 Likelihood functions

The concrete probability distribution $m(\theta)$ that a model m returns for specific parameter values θ can be compared to observed data x. This gives rise to a likelihood $\mathcal{L}_{m,x}(\theta) = \mathrm{PDF}(m(\theta), x)$ that is a real-valued function on the parameter space. Multiple distributions/models that describe different modes of observation can be combined with multiple sets of data that cover those modes of observation into a single likelihood (Sec. Likelihoods). In addition to using observed data, implementations may provide the option to use random data generated from the model (see Data) to check for Monte-Carlo closure.

1.2.6 Frequentist parameter inference

The standard method of frequentist inference is the maximum (or, respectively, profile) likelihood method. In the vast majority of cases, the test statistic used here is the likelihood ratio, that is, the ratio of two values of the likelihood corresponding to two different points in parameter space: one that maximizes the likelihood unconditionally, and one that maximizes the likelihood under some condition such as the values of the parameters of interest expected in the absence of a deviation from the null hypothesis. The corresponding building blocks for such an analysis, such as the list of parameters of interest and the likelihood function to be used, are specified in the analysis section of an HS^3 configuration (Sec. Analyses).

1.2.7 Bayesian parameter inference

The standard also encompasses the specification of Bayesian posterior distributions over parameters by combining (Sec. Analyses) likelihoods with probabilty distributions that acts the priors. Here concrete distributions are used to describe the prior probability of parameters in addition to parameterized distributions that are used to describe of the probability of observing specific data.

1.3 How to read this document

In the context of this document, any `JSON` object is referred to as a struct. A key-value-pair inside such a struct is referred to as a component. If not explicitly stated otherwise, all components mentioned are mandatory. The components located inside the top-level struct are referred to as top-level components. The

keywords OPTIONAL and REQUIRED, as well as SHOULD and MAY are used in accordance with IETF requirement levels [6].

1.4 Terms and Types

This is a list of used types and terms in this document.

- `struct`: represented with { }, containing a *key:value* mapping, keys are of type `string`.
- `component`: key-value pair within a struct
- `array` array of items (either `string`s or `number`s) without keys. Represented with `[...]`.
- `string`: references to objects, names and arbitrary information. Represented with `"..."`
- `number`: either floating or integer type values
- `boolean`: boolean values; they can be encoded as `true` and `false`

All `struct`s defining functions, distributions, parameters, variables, domains, likelihoods, data or parameter points will be referred to as `object`s. All `object`s MUST always have a component `name` that MUST be unique among all `object`s.

Within most top-level `component`s, any one `string` given as a value to any component SHOULD refer to the `name` of another `object`, unless explicitly stated otherwise. Top-level `component`s in which this is not the case are explicitly marked as such.

1.5 File format

HS^3 documents are encoded in the JSON format as defined in ISO/IEC 21778:2017 [7]. Implementations MAY support other serialization formats that support a non-ambiguous mapping to JSON, such as TOML or YAML, in which case they SHOULD use a different file extension.

The recommended filename extension for HS^3 JSON documents is `.hs3.json`.

1.6 Validators

Future versions of this standard will recommend official validator implementations and schemata. Currently, these have not been finalized.

1.7 How to get in touch

Visit the GitHub page https://github.com/hep-statistics-serialization-standard/hep-statistics-serialization-standard

2 Top-level components

In the following, the top-level `components` of HS^3 and their parameters/arguments are described. Each component is completely OPTIONAL, but certain components MAY depend on other components, which SHOULD be provided in that case. The only exception is the component `metadata` containing the version of HS^3, which is always REQUIRED. The supported top-level components are

- `distributions`: (OPTIONAL) array of `objects` defining distributions
- `functions`: (OPTIONAL) array of `objects` defining mathematical functions
- `data`: (OPTIONAL) array of `objects` defining observed or simulated data
- `likelihoods`: (OPTIONAL) array of `objects` defining combinations of distributions and data
- `domains`: (OPTIONAL) array of `objects` defining domains, describing ranges of parameters
- `parameter_points`: (OPTIONAL) array of `objects` defining parameter points. These MAY be used as starting points for minimizations or to document best-fit-values or nominal truth values of datasets
- `analyses`: (OPTIONAL) array of `objects` defining suggested analyses to be run on the models in this file
- `metadata`: REQUIRED struct containing meta information; HS^3 version number (REQUIRED), authors, paper references, package versions, data/-analysis descriptions (OPTIONAL)
- `misc`: (OPTIONAL) struct containing miscellaneous information, e.g. optimizer settings, plotting colors, etc.

In the following each of these are described in more detail with respect to their own structure.

2.1 Distributions

The top-level component `distributions` contains an array of distributions in struct format. Distributions MUST be normalized, thus, the letter $\mathcal{M}$ in the following descriptions will always relate to the normalization of the distribution. The value of $\mathcal{M}$ is conditional on the current domain. It MUST be chosen such that the integral of the distribution over the current domain equals one. The implementations might chose to perform this integral using analytical or numerical means. Each distribution MUST have the components `type`, denoting the kind of distribution described, and a component `name`, which acts as a unique identifier of this distribution among all other named objects. Distributions in general have the following keys:

- `name`: custom unique string (REQUIRED), e. g. `my_distribution_of_x`
- `type`: string (REQUIRED) that determines the kind of distribution, e. g. `gaussian_dist`
- `...`: each distribution MAY have components for the various individual parameters. For example, distributions of type `gaussian_dist` have the

specific components `mean`, `sigma` and `x`. In general, these components MAY be strings as references to other `objects`, but MAY also directly yield numeric or boolean values. Depending on the parameter and the type of distribution, they appear either in single item or array format.

An example for a two-dimensional multivariate distribution `mvnormal1` that has two free parameters `param_a` and `param_b` and might be used as a forward model - and to construct a likelihood when combined with data that has fields/columns/axes `obs_x` and `obs_y`:

Example: Distributions

```
"distributions":[
        ...
    {
        "name": "mvnormal1",
        "type": "multivariate_normal_dist",
        "mean": ["param_a", "param_b"],
        "covariances": [[1.0, 0.3], [0.3, 2.0]],
    "x": ["obs_x", "obs_y"]
  },
        ...
]
```

An example of a distribution `gauss1` that has no free parameters and might be used as a prior for a fit parameter `param_a`:

Example: Distributions

```
"distributions":[
        {
                "name":"gauss1",
                "type":"gaussian_dist",
                "mean": 4.2,
                "sigma": 1.0,
                "x":"param_a"
        },
        ...
]
```

Distributions can be treated either as `extended` or as non-`extended` [8]. Some distributions are always extended, others can never be extended, yet others can be used in extended and non-extended scenarios and have a switch selecting between the two. An non-extended distribution is always normalized to the unity. An extended distribution, on the other hand, can yield values larger than unity, where the yield is interpreted as the number of predicted events. That is to say, the distribution is augmented by a factor that is a Poisson constraint term for the total number of events. In the following, all distributions supported in HS^3 v0.2.9 are listed in detail. Future versions will expand upon this list. Entries explicitly identified as current draft spellings document ongoing post-v0.2.9 work and have not yet been assigned to a new HS^3 version.

In Cartesian products of distributions, their variate records are concatenated.

2.1.1 Univariate fundamental distributions

This section contains univariate fundamental distributions in the sense that they cannot refer to any other distribution – only to functions, parameters and exactly one variable.

2.1.1.1 Argus distribution

The Argus background distribution is defined as

$$\mathrm{ArgusPdf}(m, m_0, c, p) = \frac{1}{\mathcal{M}} \cdot m \cdot \left[1 - \left(\frac{m}{m_0}\right)^2\right]^p \cdot \exp\left[c \cdot \left(1 - \left(\frac{m}{m_0}\right)^2\right)\right]$$

and describes the ARGUS background shape.

- `name`: custom unique string
- `type`: `argus_dist`
- `mass`: name of the variable m used as mass
- `resonance`: value or name of the parameter used as resonance m_0
- `slope`: value or name of the parameter used as slope c
- `power`: value or name of the parameter used as exponent p.

2.1.1.2 Continued Poisson distribution

The continued Poisson distribution of the variable x is defined as

$$\mathrm{ContinuedPoissonPdf}(x, \lambda) = \frac{1}{\mathcal{M}} \exp\left(x \cdot \ln\lambda - \lambda - \ln\Gamma(x+1)\right),$$

where Γ denotes the Euler Gamma function.

This function is similar to the Poisson distribution (see Poisson distribution), but can accept non-integer values for x. Notably, the differences between the two might be significant for small values of x (below 1). Nevertheless, the distribution is useful to deal with datasets with non-integer event counts, such as Asimov datasets [9].

- `name`: custom unique string
- `type`: `poisson_dist`
- `x`: name of the variable x (usually referred to as k for the standard integer case)
- `mean`: value or name of the parameter used as mean λ.

2.1.1.3 Chebychev distribution

The Chebychev distribution is defined as a polynomial in Chebychev polynomials of the first kind:

$$\text{ChebychevPdf}(x, a_1, ..., a_n) = \frac{1}{\mathcal{M}} \left(T_0(z) + \sum_{i=1}^{n} a_i T_i(z) \right), \quad z = 2\frac{x - x_{\min}}{x_{\max} - x_{\min}} - 1.$$

The coefficient of the constant term $T_0(z) = 1$ is implicit and fixed to one.

- `name`: custom unique string
- `type`: `chebychev_dist`
- `x`: name of the variable x
- `coefficients`: array of coefficients a_i for the non-constant Chebychev terms. The first coefficient belongs to $T_1(z)$.

2.1.1.4 Uniform distribution

The continuous uniform distribution is defined as:

$$\text{UniformPdf}(x) = \frac{1}{\mathcal{M}}$$

- `name`: custom unique string
- `type`: `uniform_dist`
- `x`: name of the variable x

2.1.1.5 Crystal Ball distribution

The generalized Asymmetrical Double-Sided Crystal Ball line shape, composed of a Gaussian distribution at the core, connected with two power-law distributions describing the lower and upper tails, given by

$$\text{CrystalBallPdf}(m; m_0, \sigma, \alpha_L, n_L, \alpha_R, n_R) = \frac{1}{\mathcal{M}} \begin{cases} A_L \cdot (B_L - \frac{m-m_0}{\sigma_L})^{-n_L}, & \text{for } \frac{m-m_0}{\sigma_L} < -\alpha_L \\ \exp\left(-\frac{1}{2} \cdot \left[\frac{m-m_0}{\sigma_L}\right]^2\right), & \text{for } \frac{m-m_0}{\sigma_L} \leq 0 \\ \exp\left(-\frac{1}{2} \cdot \left[\frac{m-m_0}{\sigma_R}\right]^2\right), & \text{for } \frac{m-m_0}{\sigma_R} \leq \alpha_R \\ A_R \cdot (B_R + \frac{m-m_0}{\sigma_R})^{-n_R}, & \text{otherwise,} \end{cases}$$

where

$$A_i = \left(\frac{n_i}{|\alpha_i|}\right)^{n_i} \cdot \exp\left(-\frac{|\alpha_i|^2}{2}\right)$$
$$B_i = \frac{n_i}{|\alpha_i|} - |\alpha_i|$$

The keys are

- `name`: custom string
- `type`: `crystalball_dist`
- `m`: name of the variable m
- `m0`: name or value of the central value m_0
- `alpha`: value or names of α_L and α_R from above. MUST NOT be used in conjuction with `alpha_L` or `alpha_R`.
- `alpha_L`: value or names of α_L from above. MUST NOT be used in conjuction with `alpha`.
- `alpha_R`: value or names of α_R from above. MUST NOT be used in conjuction with `alpha`.
- `n`: value or names of n_L and n_R from above. MUST NOT be used in conjuction with `n_L` or `n_R`.
- `n_L`: value or names of n_L from above. MUST NOT be used in conjuction with `n`.
- `n_R`: value or names of n_R from above. MUST NOT be used in conjuction with `n`.
- `sigma`: value or names of σ_L and σ_R from above. MUST NOT be used in conjuction with `sigma_L` or `sigma_R`.
- `sigma_L`: value or names of σ_L from above. MUST NOT be used in conjuction with `sigma`.
- `sigma_R`: value or names of σ_R from above. MUST NOT be used in conjuction with `sigma`.

2.1.1.6 Decay distribution

The decay distribution describes a single- or double-sided exponential decay convolved with a resolution model R:

$$\text{DecayPdf}(t, \tau, R) = \frac{1}{\mathcal{M}} \left(B_{\text{type}} * R\right)(t),$$

where

$$B_{\text{type}}(t) = \begin{cases} \exp(-t/\tau), & \text{for SingleSided,} \\ \exp(t/\tau), & \text{for Flipped,} \\ \exp(-|t|/\tau), & \text{for DoubleSided.} \end{cases}$$

- `name`: custom unique string
- `type`: `decay_dist`
- `t`: name of the decay-time variable t
- `tau`: value or name of the decay constant τ
- `resolutionModel`: name of the resolution model R
- `decayType`: integer selecting the decay basis, with `0` for `SingleSided`, `1` for `DoubleSided`, and `2` for `Flipped`.

2.1.1.7 Exponential distribution

The exponential distribution is defined as

$$\text{ExponentialPdf}(x, c) = \frac{1}{\mathcal{M}} \cdot \exp(-c \cdot x)$$

- `name`: custom unique string
- `type`: `exponential_dist`
- `x`: name of the variable x
- `c`: value or name of the parameter used as coefficient c.

2.1.1.8 Gaussian/normal distribution

The Gaussian/normal distribution is defined as

$$\text{GaussianPdf}(x, \mu, \sigma) = \frac{1}{\mathcal{M}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- `name`: custom unique string
- `type`: `gaussian_dist` *or* `normal_dist`
- `x`: name of the variable x
- `mean`: value or name of the parameter used as mean value μ
- `sigma`: value or name of the parameter encoding the standard deviation σ.

2.1.1.9 Generalized Normal distribution

The generalized normal distribution is defined as

$$\text{GeneralizedNormalPdf}(x, \mu, \alpha, \beta) = \frac{1}{\mathcal{M}} \exp\left\{-\left(\frac{|x-\mu|}{\alpha}\right)^{\beta}\right\}$$

cf. the Wikipedia definition.

This form is of particular use with values of $\beta \sim 8$ to generate a flat-topped distribution useful for "2 point" parameter-constraints between models where there is no clear preference but their average should not be particularly favoured. (In such usage, the template points should ideally be located at $|x| < \pm 1$, to avoid disfavouring a pure solution.)

- `name`: custom unique string
- `type`: `generalized_normal_dist`
- `x`: name of the variable x
- `mean`: value or name of the parameter used as mean value μ
- `alpha`: value or name of the α parameter encoding the width; this corresponds to $\sigma = \alpha/\sqrt{2}$ when $\beta = 2$.

- `beta`: power β to which the exponent is raised, $\beta = 2$ corresponding to the GaussianPdf shape.

2.1.1.10 Log-Normal distribution

The log-normal distribution is defined as

$$\text{LogNormalPdf}(x, \mu, \sigma) = \frac{1}{\mathcal{M}} \frac{1}{x} \exp\left(-\frac{(\ln(x) - \mu)^2}{2\sigma^2}\right)$$

- `name`: custom unique string
- `type`: `lognormal_dist`
- `x`: name of the variable x
- `mu`: value or name of the parameter used as μ
- `sigma`: value or name of the parameter σ describing the shape

2.1.1.11 Landau distribution

The Landau distribution describes fluctuations in the energy loss of a charged particle traversing a thin layer of material. In terms of the conventional Landau density L, it is

$$\text{LandauPdf}(x, \mu, \sigma) = \frac{1}{\mathcal{M}} L(x; \mu, \sigma).$$

- `name`: custom unique string
- `type`: `landau_dist`
- `x`: name of the variable x
- `mean`: value or name of the location parameter μ
- `sigma`: value or name of the scale parameter σ

2.1.1.12 Poisson distribution

The Poisson distribution of the variable x is defined as

$$\text{PoissonPdf}(x, \lambda) = \frac{1}{\mathcal{M}} \frac{\lambda^x}{x!} \mathrm{e}^{-\lambda}.$$

where x is required to be an integer. In this case, the behavior for non-integer values of x is undefined.

- `name`: custom unique string
- `type`: `poisson_dist`
- `x`: name of the variable x (usually referred to as k for the standard integer case)
- `mean`: value or name of the parameter used as mean λ.

2.1.1.13 Polynomial distribution

The polynomial distribution is defined as

$$\text{PolynomialPdf}(x, a_0, a_1, a_2, ...) = \frac{1}{\mathcal{M}} \sum_{i=0}^{n} a_i x^i = a_0 + a_1 x + a_2 x^2 + ...$$

- `name`: custom unique string
- `type`: `polynomial_dist`
- `x`: name of the variable x
- `coefficients`: array of coefficients a_i. The length of this array implies the degree of the polynomial.

2.1.2 Multivariate fundamental distributions

This section contains multivariate fundamental distributions. They may refer to functions, parameters and more than one variable.

2.1.2.1 Barlow-Beeston-Lite Constraint distribution

This distribution represents a product of Poisson distributions defining the statistical uncertainties of the histogram templates defined in a `histfactory` function.

$$\text{BarlowBeestonLitePoissonConstraintPdf}(x) = \frac{1}{\mathcal{M}} \prod_{i}^{n} \text{PoissonPdf}(x_i \cdot \tau_i, \tau_i)$$

- `name`: custom unique string
- `type`: `barlow_beeston_lite_poisson_constraint_dist`
- `x`: array of names of the variables x_i. This also includes mixed arrays of values and names.
- `expected`: array of central values τ_i

2.1.2.2 Multivariate normal distribution

The multivariate normal distribution is defined as

$$\text{MvNormalPdf}(\mathbf{x}, \mu, \Sigma) = \frac{1}{\mathcal{M}} \exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^\intercal \Sigma^{-1} (\mathbf{x}-\mu)\right),$$

with $\Sigma \in \mathbb{R}^{k \times k}$ being positive-definite.

- `name`: custom unique string
- `type`: `multivariate_normal_dist`
- `x`: array of names of the variables $\mathbf{x}$. This also includes mixed arrays of values and names.
- `mean`: array of length k of values or names of the parameters used as mean values μ

- `covariances`: an array comprised of k sub-arrays, each of which is also of length k, designed to store values or names of the entries of the covariance matrix Σ. In general, the covariance matrix Σ MUST be symmetric and positive semi-definite.

2.1.2.3 Generic distribution

Note: Users should prefer the specific distributions defined in this standard over generic distributions where possible, as implementations of these will typically be more optimized. Generic distributions should only be used if no equivalent specific distribution is defined.

A generic distribution is defined by an expression that represents the PDF of the distribution in respect to the Lebesgue measure. The expression must be a valid HS3-expression string (see Section Generic Expressions).

- `name`: custom string
- `type`: `generic_dist`
- `expression`: a string with a generic mathematical expression. Simple mathematical syntax common to most programming languages should be used here, such as `x-2*y+z`. The arguments `x`, `y` and `z` in this example MUST be parameters, functions or variables. The distribution is normalized by the implementation, so a normalization term SHOULD NOT be included in the expression. If the expression results in a negative value, the behavior is undefined.

2.1.2.4 HistFactory distribution

A HistFactory distribution is a joint probability distribution over bin counts n_i, with independent Poisson probabilities whose expectations are supplied by a mixture of histogram templates. Its random variables are the bin counts, not the observable x used to define the fixed bins. The corresponding `histfactory` function (see Section Functions) provides the expected bin yields $\lambda_i(\theta)$ from the sample mixture and its modifiers:

$$\text{HistFactoryPdf}(\mathbf{n} \mid \theta) = \prod_i \text{PoissonPdf}(n_i, \lambda_i(\theta))$$

This probability mass function is normalized over nonnegative integer count vectors $\mathbf{n}$. The parameters θ may include both parameters of interest and nuisance parameters.

The prediction $\lambda_i(\theta)$, the modifiers and their constraints, and an example are defined in Section HistFactory.

- `name`: custom unique string
- `type`: `histfactory_dist`
- all further components as defined in Section HistFactory

2.1.2.5 Relativistic Breit-Wigner distribution

The relativistic Breit-Wigner distribution describes the line-shape of a resonance in the mass spectrum of a two-particle system. It is assumed that the resonance can decay into a list of channels.

The first channel in the list indicates the system for which mass distribution is modelled.

$$\begin{aligned}\text{BreitWignerPDF}(m, m_{\text{BW}}) &= \frac{1}{\mathcal{M}} \frac{m\Gamma_1(m)}{\left|m_{\text{BW}}^2 - m^2 - im_{\text{BW}}\Gamma(m)\right|^2}, \\ \Gamma(m) &= \sum_i \Gamma_i(m),\end{aligned}$$

When modelling the mass spectrum, the term m in the numerator of Eq. (Breit-Wigner) accounts for a jacobian of transformation from m^2 to m. The width term $\Gamma_1(m)$ adds for the phase space factor for the channel of interest

- `name`: custom unique string
- `type`: `relativistic_breit_wigner_dist`
- `mass`: name of the mass variable m_{BW}
- `channels`: list of `structs` encoding the channels

Each of the channels is defined by the partial width $\Gamma_i(m)$, given as

$$\begin{aligned}\Gamma_i(m) &= \Gamma_{\text{BW},i} n_{li}^2(m) \rho_i(m), \\ \rho_i(m) &= 2q_i(m)/m, \\ q_i(m) &= \sqrt{(m^2 - (m_{1i} + m_{2i})^2)(m^2 - (m_{1i} - m_{2i})^2)}/(2m), \\ n_{li}(m) &= z_i^{li}(m) h_{li}(z_i(m)), \\ z_i(m) &= q_i(m) R_i\end{aligned}$$

The $h_l(z)$ is the standard Blatt-Weisskopf form-factors, $h_0^2(z) = 1/(1+z^2)$, $h_1^2(z) = 1/(9+3z^2+z^4)$, and so on (Eqs.(50.30-50.35) in Ref. [10]). The `structs` defining the channels should contain the following keys:

- `name`: name of the final state (OPTIONAL)
- `Gamma`: partial width Γ_{BW} of the resonance
- `m1`: mass m_1 of the first particle the resonance decays into (default value 0)
- `m2`: mass m_2 of the second particle the resonance decays into (default value 0)
- `l`: orbital angular momentum l (default value 0)
- `R`: form-factor size parameter R (default value 3 GeV) For non-zero angular momentum, $\Gamma_i(m_{\text{BW}})$ gives an approximation to the partial width of the

resonance, not $\Gamma_{\mathrm{BW},i}$. A commonly used approximation of the relativistic Breit-Wigner function with the constant width is a special case of the Eq. (Breit-Wigner), where the `[channels]` argument contains a single channel with $m_1 = 0$, $m_2 = 0$, and $l = 0$.

2.1.3 Composite distributions

This section contains composite distributions in the sense that they refer to other distribution which they combine or modify in some way.

2.1.3.1 Mixture distribution

The mixture distribution, sometimes called addition of distributions, is a (weighted) sum of distributions $f_i(\vec{x})$, depending on the same variable(s) $\vec{x}$:

$$\mathrm{MixturePdf}(x) = \frac{1}{\mathcal{M}} \sum_{i=1}^{n} c_i \cdot f_i(\vec{x}),$$

where the c_i are coefficients and $\vec{x}$ is the vector of variables.

- `name`: custom unique string
- `type`: `mixture_dist`
- `x`: name of the variable x (OPTIONAL, since the variable is fully defined by the summands)
- `summands`: array of names referencing distributions
- `coefficients`: array of names of coefficients c_i or numbers to be added
- `extended`: boolean denoting whether this is an extended distribution (OPTIONAL, as it can be inferred from the lengths of the lists for `summands` and `coefficients`) This distribution can be treated as extended or as non-extended.
- If the number of coefficients equals the number of distributions and `extended` is `false`, then the sum of the coefficient must be (approximately) one.
- If the number of coefficients equals the number of distributions and `extended` is `true`, then the sum of the coefficients will be used as the Poisson rate parameter and the mixture distribution itself will use the coefficients normalized by their sum.
- If the number of coefficients is one less than the number of distributions, then `extended` must be `false` and c_n is computed from the other coefficients as

$$c_n = 1 - \sum_{i=1}^{n-1} c_i.$$

2.1.3.2 Mixture model

The mixture model is a weighted sum of resolution models or model PDFs:

$$\text{MixtureModel}(x) = \sum_{i=1}^{n} c_i \, m_i(x).$$

If one coefficient fewer than the number of summands is provided, the final coefficient is computed as

$$c_n = 1 - \sum_{i=1}^{n-1} c_i.$$

If all coefficients are given, their sum is used as the expected number of events in extended use cases.

- `name`: custom unique string
- `type`: `mixture_model`
- `summands`: array of names referencing model distributions
- `coefficients`: array of names of coefficients c_i or numbers
- `extended`: boolean denoting whether this is an extended model

2.1.3.3 Product distribution

The product of independent distributions f_i is defined as

$$\text{ProductPdf}(x) = \frac{1}{\mathcal{M}} \prod_{i}^{n} f(x).$$

- `name`: custom string
- `type`: `product_dist`
- `x`: name of the variable x (OPTIONAL, since the variable is fully defined by the factors)
- `factors`: array of names referencing distributions

2.1.3.4 Efficiency product distribution

The efficiency product distribution multiplies a normalized PDF with an efficiency function and normalizes the product:

$$\text{EfficiencyProductPdf}(x) = \frac{1}{\mathcal{M}} \, f(x) \, \epsilon(x).$$

The efficiency function is expected to be non-negative over the domain.

- `name`: custom unique string

- `type`: `efficiency_product_pdf_dist`
- `pdf`: name of the input distribution f
- `eff`: name of the efficiency function ϵ

2.1.3.5 Density Function distribution

A distribution that is specified via a non-normalized density function $f(x)$. Formally, it corresponds to the probability measure that has the density $f(x)$ in respect to the Lebesgue measure.

The density function is normalized automatically by $\mathcal{M} = \int f(x)dx$, so the PDF of the distribution is

$$\text{DensityFunctionPdf}(f, x) = \frac{f(x)}{\mathcal{M}}$$

The distribution can be specified either via the non-normalized density function $f(x)$ or via the non-normalized log-density function $\log(f(x))$:

- `density_function_dist`: Specified via the PDF $f(x)$
  - `name`: custom string
  - `type`: `density_function_dist`
  - `function`: The density function f
- `log_density_function_dist`: Specified via the PDF $f(x)$
  - `name`: custom string
  - `type`: `log_density_function_dist`
  - `function`: The density function f

2.1.3.6 Poisson point process

A Poisson point process distribution is understood here as the distribution of the outcomes of a (typically inhomogeneous) Poisson point process. In particle physics, such a distribution is often called an extended distribution [8]. In HS3, a Poisson point process distribution is specified using a global-rate parameter λ and an underlying distribution m, either explicitly or implicitly (see below). Random values are drawn from the Poisson point process distribution by drawing a random number n from a Poisson distribution of the global-rate λ, and then drawing n random values from the underlying distribution m. The resulting random values are vectors x of length n, so their length varies. The PDF the Poisson point process distribution at an outcome x (a vector of length n) is

$$\text{PoissonPointProcessPdf}(\lambda, m, x) = \text{PoissonPdf}(n, \lambda) \cdot \prod_{i}^{n} \text{PDF}(m, x_i).$$

In particle physics, the function $\text{PoissonPointProcessPdf}(\lambda, m(\theta), x)$, for a fixed observation x and varying parameters λ and θ, is often called an extended

likelihood. A Poisson point process distribution can be specified in HS3 in two ways:

- `rate_extended_dist`: The global-rate parameter λ and underlying distribution m are specified explicitly:
  - `name`: custom string
  - `type`: `rate_extended_dist`
  - `rate`: The global rate λ
  - `dist`: The underlying distribution m

The name of the variable x is taken from the underlying distribution. The underlying distribution MUST not be referred to from other components of the statistical model.

- `rate_density_dist`: Specified via a non-normalized rate-density function f. Both the global-rate parameter and the underlying distribution are implicit: $\lambda = \int f(y)dy$ and $m = \text{DensityFunctionPdf}(f)$. More formally, the distribution corresponds to the inhomogeneous Poisson point process that is defined by a non-normalized rate measure which has density f in respect to the Lebesgue measure.
  - `name`: custom string
  - `type`: `rate_density_dist`
  - `x`: name of the variable x
  - `density`: The rate-density function f

2.1.3.7 Bin-count distribution

This is a binned version of the Poisson point process distribution (Poisson point process). It is the distribution of the bin counts that result from histogramming the outcomes of a Poisson point process distribution using a given binning scheme (see Binned Data). Like the Poisson point process distribution, a Bin-counts distribution can either be specified via a global-rate parameter λ and an underlying distribution m, or via a rate-density function f (in which case λ and m are implicit). In addition, the binning scheme also has to be specified in either case, unless it can be inferred (see below). For k bins, this type of distribution corresponds to a product of k Poisson distributions with rates

$$\nu_i = \lambda \cdot \int_{\text{bin}_i} \text{PDF}(y, m)dy \quad \text{equivalent to}$$
$$\nu_i = \int_{\text{bin}_i} f(y)dy$$

The PDF of the bin-counts distribution at an outcome x (a vector of length k, same as the number of bins) is

$$\text{BinCountsPdf}(x) = \prod_i^k \text{PoissonPdf}(x_i, \nu_i)$$

- `bincounts_extended_dist`: The global-rate parameter λ and underlying distribution m are specified explicitly:
  - `name`: custom string
  - `type`: `bincounts_extended_dist`
  - `rate`: The global rate λ
  - `dist`: The underlying distribution m
  - `axes`: a definition of the binning to be used, following the in Binned Data. OPTIONAL if `dist` is a binned distribution, in which case the same binning is used by default.

The name of the variable x is taken from the underlying distribution. The underlying distribution MUST not be referred to from other components of the statistical model. `bincounts_density_dist`: Specified via a non-normalized rate-density function f. Both the global-rate parameter and the underlying distribution are implicit: $\lambda = \int f(y)dy$ and $m = \text{DensityFunctionPdf}(f)$.

More formally, the distribution corresponds to the inhomogeneous Poisson point process that is defined by a non-normalized rate measure which has density f in respect to the Lebesgue measure.

- `name`: custom string
  - `type`: `bincounts_density_dist`
  - `x`: name of the variable x
  - `density`: The rate-density function f
  - `axes`: a definition of the binning to be used, following the definitions in Binned Data.

2.2 Functions

The top-level component `functions` describes an array of mathematical functions in struct format to be used as helper objects. Similar to distributions each entry is REQUIRED to have the components `type` and `name`. Other components are dependent on the kind of functions. The field `name` is REQUIRED and may be any custom unique string. Functions in general have the following components:

- `name`: custom unique string
- `type`: string that determines the kind of function, e.g. `sum`
- `...`: each function has individual parameter keys for the various individual parameters. For example, functions of type `sum` have the parameter key `summands`. In general, these keys can describe strings as references to other objects or numbers. Depending on the parameter and the type of function, they appear either in single item or array format.

Example: Functions

```
"functions": [
```

```
        {
                "name" : "sum1",
                "type" : "sum",
                "summands" : [1.8, 4, "param_xy"]
        },
        ...
]
```

In the following the implemented functions are described in detail.

2.2.1 Product

A product of values or functions a_i.

$$\text{Prod} = \prod_i^n a_i$$

- `name`: custom unique string
- `type`: `product`
- `factors`: array of names of the elements of the product or numbers.

2.2.2 Sum

A sum of values or functions a_i.

$$\text{Sum} = \sum_i^n a_i$$

- `name`: custom unique string
- `type`: `sum`
- `summands`: array of names of the elements of the sum or numbers.

2.2.3 Polynomial Function

The polynomial function is defined as

$$\text{Polynomial}(x, a_0, a_1, \dots, a_n) = \sum_{i=0}^{n} a_i x^i.$$

Unlike `polynomial_dist`, this object is a function and is not normalized.

- `name`: custom unique string
- `type`: `polynomial`
- `x`: name of the variable x
- `coefficients`: array of coefficients a_i. The length of this array implies the degree of the polynomial.

2.2.4 Derivative Function

The derivative function represents a numerical derivative of another function with respect to a variable:

$$\text{Derivative}(f, x, n) = \frac{d^n f}{dx^n}.$$

The order is intended for first, second, or third derivatives, and `eps` controls the finite-difference step size.

- `name`: custom unique string
- `type`: `derivative`
- `function`: name of the function f
- `x`: name of the differentiation variable x
- `order`: derivative order n
- `eps`: step size used for the numerical derivative
- `normalization`: array of names defining the normalization set (OPTIONAL)

2.2.5 Gaussian Resolution Model

The Gaussian resolution model represents a Gaussian smearing model in the convolution variable x:

$$\text{GaussModel}(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

It can be used as a resolution model for analytical convolutions.

- `name`: custom unique string
- `type`: `gauss_model_function`
- `x`: name of the convolution variable x
- `mean`: value or name of the mean parameter μ
- `sigma`: value or name of the width parameter σ

2.2.6 Truth Resolution Model

The truth resolution model corresponds to perfect resolution, represented by a delta function in the convolution variable:

$$\text{TruthModel}(x) = \delta(x).$$

When used in an analytical convolution, it returns the unconvolved basis function.

- `name`: custom unique string
- `type`: `truth_model_function`
- `x`: name of the convolution variable x

2.2.7 Integral Function

The integral function represents the integral of a function f over one or more variables:

$$\text{Integral}(f, \vec{x}) = \int f(\vec{x})\, d\vec{x}.$$

If `domain` is present, the integral uses the named domain for the integration variables. If `normalization` is present, the integrand is evaluated with that normalization set.

- `name`: custom unique string
- `type`: `integral`
- `integrand`: name of the function f to integrate
- `variables`: array of names of the integration variables $\vec{x}$
- `domain`: name of the integration domain (OPTIONAL)
- `normalization`: array of names defining the normalization set (OPTIONAL)

2.2.8 HistFactory Function

A HistFactory function provides the expected counts in fixed bins of an observable x by adding sample histogram yields with their modifiers (normalization factors, systematic variations, statistical uncertainties). It is the deterministic prediction component of the HistFactory model:

$$\text{HistFactoryFunction}_i(\theta) = \lambda_i(\theta)$$

Here i labels a fixed bin in x; θ controls the sample yields and shapes. The corresponding `histfactory_dist` uses these expectations to model the random bin counts n_i. The mixture interpretation, parameter roles, modifiers and example are defined in Section HistFactory.

- `name`: custom unique string
- `type`: `histfactory`
- all further components as defined in Section HistFactory

2.2.9 Generic Function

Note: Users should prefer the specific functions defined in this standard over generic functions where possible, as implementations of these will typically be more optimized. Generic functions should only be used if no equivalent specific distribution is defined. A generic function is defined by an expression. The expression must be a valid HS3-expression string (see Section Generic Expressions).

- `name`: custom unique string

- `type`: `generic` (current draft spelling; `generic_function` is the legacy HS^3 v0.2.9 key)
- `expression`: a string with a generic mathematical expression. Simple mathematical syntax common to programming languages should be used here, such as `x-2*y+z`. For any non-elementary operations, the behavior is undefined.

2.3 Data

The top-level component `data` contains an array of data sets in struct format. Each data set needs to contain the components `type` and `name`. Other components are dependent on the type of data set as demonstrated below:

- `name`: custom string
- `type`: string that determines the format of the observations
- `...`: each type of observations has different parameter keys. Some of these are OPTIONAL and marked accordingly in the more detailed description below

A detailed description of the different types with examples can be found below. While data, in most settings, has no uncertainty attached to it, this standard *does* allow to provide data with uncertainty, as there are some settings in which uncorrelated or correlated errors need to be taken into account. These include cases such as

- generated data obtained from importance sampling, including a (potentially Gaussian) uncertainty from the frequency weights
- unfolded data, resulting from arbitrarily complex transformation functions involving statistical models folding some degree of uncertainty into the data points themselves

While it should always be preferred to publish "raw" data, allowing to include pre-processed data with corresponding uncertainties expands the possible applications considerably.

2.3.1 Axis Specifications

Axes define the observable domain for data and PDFs. Every axis is either continuous or categorical. The kind of an axis is determined structurally: an axis struct MUST contain exactly one of the field pair `min`/`max` (continuous axis) or the field `categories` (categorical axis). Continuous axes additionally have type-specific requirements based on the data type they describe.

2.3.1.1 Continuous Axes

A continuous axis describes a real-valued observable. All continuous axes MUST include:

- `name`: identifier matching the observable in the PDF

- `min`: lower bound of the observable domain
- `max`: upper bound of the observable domain

These fields define the domain over which PDFs are normalized.

Note on normalization: Any PDF depending on a continuous observable x must be normalized over the domain $[\min, \max]$ defined by that observable's axis, not over $(-\infty, \infty)$.

2.3.1.2 Categorical Axes

A categorical axis describes a discrete-valued observable that takes one of a finite set of named states. A categorical axis MUST include:

- `name`: identifier matching the categorical observable in the PDF
- `categories`: array of unique strings enumerating the allowed values. `categories` defines a set: the order in which values appear in the array MUST NOT affect evaluation.

The `categories` array itself defines the domain of the axis; the fields `min`, `max`, `nbins` and `edges` MUST NOT be present.

2.3.1.3 Forbidden Attributes

The `const` attribute MUST NOT appear in axis specifications. All axes define observable domains, never fit parameters.

2.3.1.4 Axis Types by Data Type

For Point Data:

- Only continuous axes are permitted, carrying only the base fields (`name`, `min`, `max`)
- Binning fields (`nbins`, `edges`) MUST NOT be present

For Unbinned Data:

- Continuous axes are required to carry base fields (`name`, `min`, `max`)
- Binning fields (`nbins`, `edges`) MUST NOT be present
- Categorical axes are permitted; the corresponding column of `entries` holds the respective row's category as one of the strings listed in `categories`

For Binned Data:

- Continuous axes are required to carry base fields (`name`, `min`, `max`)
- Each continuous axis MUST carry exactly one binning specification:
  1. Regular binning: `nbins` (integer number of equal-width bins)
  2. Irregular binning: `edges` (array of length $n + 1$ bin boundaries)
- Both `nbins` and `edges` MUST NOT be specified simultaneously
- Neither `nbins` nor `edges` MUST NOT be omitted on a continuous axis

- Categorical axes are permitted. A categorical axis acts as its own binning, with one bin per category in the order given by `categories`

For irregular binning, the `edges` array must satisfy: - `edges[0] == min` - `edges[-1] == max` - All values must be in strictly ascending order

2.3.2 Point Data

Point data describes a measurement of a single number, with a possible uncertainty (error).

- `name`: custom string
- `type`: `point`
- `value`: value of this data point
- `axes`: (OPTIONAL) array of axis structs. Each struct must contain `name`, `min`, `max` as defined in Axis Specifications. When present, associates the point measurement with a specific observable domain.
- `uncertainty`: (OPTIONAL) uncertainty of this data point

Example: Point Data

```
"data":[
        {
                "name":"data1",
                "type":"point",
                "value":0.,
                "uncertainty":1.,
                "axes":[
                        { "name":"obs_x", "min":-5, "max":5 }
                ]
        }
]
```

2.3.3 Unbinned Data

Unbinned data describes a measurement of multiple data points in a possibly multi-dimensional space of variables. These data points can be weighted.

- `name`: custom string
- `type`: `unbinned`
- `entries`: array of arrays containing the coordinates/entries of the data
- `axes`: array of axis structs. Each struct MUST be a valid continuous or categorical axis as defined in Axis Specifications. Binning fields (`nbins`, `edges`) MUST NOT be present.
- `weights`: (OPTIONAL) array of values containing the weights of the individual data points, to be used for χ^2 comparisons and fits. If this component is not given, weight 1 is assumed for all data points. If given, the array needs to be of the same length as `entries`.

- `entries_uncertainties`: (OPTIONAL) array of arrays containing the errors/uncertainties of each entry. If given, the array needs to be of the same shape as `entries`.

Example: Unbinned Data

```
"data":[
  {
    "name":"data1",
    "type":"unbinned",
    "weights":[ 9.0, 18.4 ],
    "entries":[ [1,3], [2,9] ],
    "entries_uncertainties":[ [0.3], [0.6] ],
    "axes":[
      { "name":"variable1", "min":1, "max":3 },
      { "name":"variable2", "min":-10, "max":10 }
    ]
  }
]
```

Unbinned data sets with a categorical axis store the category string in the corresponding column of `entries`:

Example: Unbinned Data with categorical axis

```
"data":[
  {
    "name":"data2",
    "type":"unbinned",
    "weights":[12.0, 4.3, 2.1],
    "entries":[["SR",7,0.1], ["SR",9,-0.2], ["CR",4,0.9]],
    "axes":[
      {"name":"cat1", "categories":["SR","CR"]},
      {"name":"variable1", "min":3, "max":20},
      {"name":"variable2", "min":-1, "max":1}
    ]
  }
]
```

Here the first column of `entries` refers to the `cat1` axis, giving the category of each row directly as `"SR"` or `"CR"`.

2.3.4 Binned Data

Binned data describes a histogram of data points with bin contents in a possibly multi-dimensional space of variables. Whether entries that fall precisely on the bin boundaries are sorted into the smaller or larger bin is under the discretion of the creator of the model and thus not defined.

- `name`: custom string
- `type`: `binned`

- `contents`: array of values representing the contents of the binned data set
- `axes`: array of axis structs. Each struct MUST be a valid continuous or categorical axis as defined in Axis Specifications. Each continuous axis MUST carry exactly one binning specification:
  1. Regular binning: `nbins` (number of equal-width bins)
  2. Irregular binning: `edges` (array of length $n + 1$ bin boundaries)
- `uncertainty`: (OPTIONAL) struct representing the uncertainty of the contents. It consists of up to three components:
  - `type`: denoting the kind of uncertainty, for now only Gaussian distributed uncertainties denoted as `gaussian_uncertainty` are supported
  - `sigma`: array of the standard deviation of the entries in `contents`. Needs to be of the same length as `contents`
  - `correlation`: (OPTIONAL) array of arrays denoting the correlation between the contents in matrix format. Must be of dimension length of `contents` $\times$ length of `contents`. It can also be set to 0 to indicate no correlation.

Example: Binned Data

```
"data":[
  {
    "name":"data3",
    "type":"binned",
    "contents":[ 9.0, 18.4 ],
    "axes":[ { "name":"variable1", "nbins":2, "min":1, "max":3 } ]
  },
  {
    "name":"asimov_data3",
    "type":"binned",
    "contents":[ 9.0, 18.4, 13, 0. ],
    "axes":[
      { "name":"variable1", "nbins":2, "min":1, "max":3 },
      { "name":"variable2", "min":0, "max":100, "edges":[0,10,100] }
    ]
  }
]
```

The `contents` of the data sets are represented in a flattened one-dimensional array of bin-contents. Here the behaviour is the same for a categorical axis with n categories as for regularly-binned continuous axis with n bins: each category corresponds to one bin along the respective dimension of the flattened `contents` array.

Example: Binned Data with categorical axis

```
"data":[
  {
    "name":"data4",
    "type":"binned",
```

```
    "content":[9.0, 8.4, 4.0, 7.0],
    "axes":[
      {"name":"channel", "categories":["SR","CR"]},
      {"name":"variables1", "nbins":2, "min":0, "max":10}
    ]
  }
]
```

This type can also be used to store pre-processed data utilizing the `uncertainty` component

Example: Pre-processed binned Data

```
"data":[
  {
    "name":"data5",
    "type":"binned",
    "contents":[ 9.0, 18.4 ],
    "uncertainty" : {
      "type": "gaussian_uncertainty",
      "correlation" : 0,
      "sigma" : [ 3, 4 ]
     },
    "axes":[
      { "name":"variable1", "nbins":2, "min":1, "max":3 }
    ]
  }
]
```

2.4 Likelihoods

The top-level component `likelihoods` contains an array of likelihoods in struct format specifying mappings of distributions and observations. The corresponding distributions and observations are inserted as keys in string format referencing to distributions and observations defined in the respective top-level components, or as numbers for fixed data values. The combination of parameterized distributions $m_i(\theta_i)$ with observations x_i generates a likelihood function

$$\mathscr{L}(\theta_1, \theta_2, ...) = \prod_i \mathrm{PDF}(m_i(\theta_i), x_i)$$

The components of a likelihood struct are:

- `name`: custom string
- `distributions`: array of strings referencing the considered distributions
- `data`: array of strings referencing the used data, must be of the same length as the array of `distributions`. Alternatively, the data-values for single-dimensional distributions can be given in-line. For example, this can be used for constraint terms representing auxiliary measurements.

- `aux_distributions`: (OPTIONAL) array of strings referencing the considered auxiliary distributions defined in the top-level component `distributions`. They can be used to encode regularizers or penalty terms to aid the minimization. These observed data for these distributions is implicit and not part of `data`.

Example: Likelihoods

```
"likelihoods":[
  {
    "name":"likelihood1",
    "distributions":[
      "dist1",
      "dist2",
      "single_dimensional_dist_1",
      ...
    ],
    "data":         [
      "data1",
      "data2",
      0,
      ...
    ],
    "aux_distributions" : [ "regularization_term" ]
  },
  ...
]
```

2.5 Domains

The top-level component `domains` contains an array of domains giving information on the ranges of parameters and variables in struct format. Within a specified domain, the corresponding model is expected to yield valid values. Each domain must contain a `name` and a `type` although right now only the `product_domain` type is supported, even though others like e. g. a simplex domain might be added later. A domain consists of the following components:

- `name`: custom string
- `type`: `product_domain`
- `axes`: array of parameters and variables in this domain in terms of individual axes. These can be continuous or categorical, mirroring the axis types defined in Axis Specifications.
  - A continuous axis has the components:
    - `name`: custom string
    - `max`: upper bound of range
    - `min`: lower bound of range.
  - A categorical axis has the components:
    - `name`: custom string

        * `categories`: array of unique strings enumerating the allowed values

Example: Domains

```
"domains":[
  {
    "name":"domain1",
    "type":"product_domain",
    "axes": [
      { "name" : "par_1", "min" : 1, "max" : 8 },
      { "name" : "par_2", "min" : 4.78, "max" : 6 },
      { "name" : "channel", "categories": ["SR","CR"] },
      ...
    ]
  },
  ...
]
```

2.6 Parameter points

The top-level component `parameter_points` contains an array of parameter configurations. These can be starting values for minimizations, parameter settings used to generate toy data, best-fit-values obtained, or points in parameter space used for different purposes.

- `name`: custom string
- `parameters`: array of parameter structs (see below) The component `parameters` is an array of components each containing:
  - `name`: custom string
  - `value`: number, value of variable
  - `const`: (OPTIONAL) boolean, whether variable is constant or not. Default is `false`.

Example: Parameter points

```
"parameter_points":[
  {
    "name" : "starting_values",
    "parameters": [
      { "name" : "par_1", "value": 3 },
      { "name" : "par_2", "value": 7, "const": true },
      ...
     ]
  },
  ...
]
```

2.7 Analyses

The top-level component `analyses` contains an array of possible (automated) analyses. To that extent, likelihoods, parameters of interest and the affiliated domains are listed. Description of the components:

- `name`: long custom string
- `likelihood`: name as reference to a likelihood defined in the top-level component `likelihoods`
- `parameter_of_interest`: (OPTIONAL) array of names as reference to parameters that are interesting for the analysis at hand
- `domain`: name of a domain to be used for the parameters, defined in the top-level component `domains`
- `init`: (OPTIONAL) name of an initial value to be used, defined in the top-level component `parameter_points`
- `prior`: (OPTIONAL) name of a prior distribution, defined in the top-level component `distributions`. This is only used for Bayesian interpretations and should not be confused with auxiliary distributions listed in the likelihood section. The prior could, for example, be a product distribution of all the individual priors. If for any parameter, both a prior and a parameter domain are given, the prior should be truncated to the given parameter domain. Otherwise, implicit flat priors over the given parameter domain are assumed.

All parameters of all distributions in the likelihood must either be listed under the domain referenced, or set to `const` in the parameter point referenced.

Example: Analyses

```
"analyses": [
  {
    "name" : "analysis1",
    "likelihood" : "likelihood1",
    "aux_likelihood_terms" : ["distribtion_1", "distribution_2", ...]
    "parameters_of_interest" : ["param1"],
    "domain" : "domain1" ,
    "init" : "starting_values",
    "prior" : "prior_dist"
  },
  ...
]
```

2.8 Metadata

The top-level component `metadata` contains meta-information related to the creation of the file. The component `hs3_version` stores the HS^3 version and is REQUIRED for now. Overview of the components:

- `hs3_version`: (REQUIRED) HS^3 version number as String for reference

- `packages`: (OPTIONAL) array of structs defining packages and their version number used in the creation of this file, depicted with the components `name` and `version` respectively
- `authors`: (OPTIONAL) array of authors, either individual persons, or collaborations
- `publications`: (OPTIONAL) array of document identifiers of publications associated with this file
- `description`: (OPTIONAL) short abstract/description for this file

Example: Metadata

```
"metadata" : {
  "hs3_version" : "0.2.0",
  "packages" : [ { "name": "ROOT", "version": "6.28.02" } ],
  "authors": ["The ATLAS Collaboration", "The CMS Collaboration"],
  "publications": ["doiABCDEFG"]
}
```

2.9 Miscellaneous

The top-level component `misc` can contain arbitrary, user-created information in struct format.

Example: Miscellaneous

```
"misc" : {
  "customkey1" : "custom information 1" ,
  "myPackage_internal" : {
    "somekey" : "custom info only affecting myPackage",
    "default_color_for_all_curves" : "fuchsia" }
  }
```

This top-level component is intended to store any and all additional information, including user- or backend-specific meta-information. Examples include, but are not limited to:

- colors and styles for drawing distributions in this file
- suggested settings for samplers or minimizers when working with the distributions in this file
- comments explaining design choices made when building the model in this file
- suggested names and paths for output files to be used by backends working with this file

3 Supplementary Material

This section contains supplementary material referenced in section [sec:toplevel]

3.1 Generic Expressions

This section details the HS^3 *generic expression* language. Expressions can be use to specify generic functions and generic distributions. The implementations that support generic expression MUST support:

- Literal integer (format `1234`), boolean (format `TRUE` and `FALSE`) and floating point (format `123.4`, `1.234e2` and `1.234E2`) values.
- Literal values for π (`PI`) and Euler's number (`EULER`).
- The arithmetic operators addition (`x + y`), subtraction (`x - y`), multiplication (`x * y`), division (`x / y`) and exponentiation (`x^y`).
- The comparison operators approximately-equal (`x == y`), exactly-equal (`x === y`), not-approximately-equal (`x != y`), not-exactly-equal (`x !== y`), less-than (`x < y`), less-or-equal (`x <= y`), greater-than (`x > y`) and greater-or-equal (`x >= y`).
- The logical operators logical-inverse (`!a`), logical-and (`a && b`) and logical-or (`a || b`).
- Round brackets to specify the order of operations.
- The conditional expression `ifelse(condition, result_if_true, result_if_false)`.
- The functions
  - `exp(x)`: Euler's number raised to the power of `x`
  - `log(x)`: Natural logarithm of `x`
  - `sqrt(x)`: The square root of `x`
  - `abs(x)`: The absolute value of `x`
  - `floor(x)`: The greatest integer less than or equal to `x`
  - `ceil(x)`: The smallest integer greater than or equal to `x`
  - `pow(x, y)`: `x` raised to the power of `y`
  - `min(x, y)`: minimum of `x` and `y`
  - `max(x, y)`: maximum of `x` and `y`
  - `sin(x)`: The sine of `x`
  - `cos(x)`: The cosine of `x`
  - `tan(x)`: The tangent of `x`
  - `asin(x)`: The inverse sin of `x`
  - `acos(x)`: The inverse cosine of `x`
  - `atan(x)`: The inverse tangent of `x`
  - `erf(x)`: The standard error function of `x`

Symbols not defined here refer to variables in the HS^3 model. The operator precedence and associativity is according to the common conventions. Spaces between operators and operands are optional. There must be no space between a function name and the function arguments, spaces between function arguments are optional (`f(a, b)` and `f(a,b)` are correct but `f (a, b)` is not allowed).

Division MUST be treated as floating-point division (i.e. `2/3` should be equivalent to `2.0/3.0`).

The approximately-equal (`a == b`) and the not-approximately-equal operator (`a != b`), SHOULD compare floating-point numbers to within a small multiple of

the unit of least precision.

Conditional expression may be evaluated in a lazy fashion (evaluating either `result_if_true` or `result_if_false` depending on the result of `condition`) or in an eager fashion (evaluating all of `condition`, `result_if_true` and `result_if_false` in arbitrary order). Evaluation strategy is left to the engine. This implies that on some engines, large trees of conditional expressions may have a high evaluation cost.

The behavior of any functions and operators not listed above is not defined, they are reserved for future versions of this standard. Implementations MAY support additional functions and operators as experimental features, but their use is considered non-standard and results in non-portable and potentially non-forward-compatible HS^3 documents.

3.2 Authoring Requirements for HS^3 Primitive Specifications

Primitive specifications define the mathematical semantics of new HS^3 primitives. Their purpose is to enable independent implementations to interpret serialized models consistently and without ambiguity.

3.2.1 Intended audience

Primitive specifications SHOULD be understandable to three audiences simultaneously:

- **Software implementers**, who should be able to implement import, export, and evaluation directly from the specification.
- **Domain experts**, who should recognize how concepts from existing software correspond to the HS^3 representation.
- **Statisticians**, who should be able to infer the mathematical properties of the primitive directly from its specification and serialized representation.

The mathematical definition is normative. Implementation-specific terminology may be used where it improves understanding, but SHOULD complement rather than replace the mathematical semantics.

3.2.2 Design principles

A primitive specifies **what** a model computes, not **how** a particular implementation computes it.

Specifications shall therefore define primitives in mathematical terms whenever practical. Implementation-specific concepts may be included only where they are necessary to faithfully represent existing software.

A primitive that introduces functionality which cannot already be represented

using existing HS^3 primitives will generally be appropriate, provided its semantics are well defined.

A primitive that only specializes an existing abstraction SHOULD provide clear additional value, such as a substantially more compact representation of a common use case, access to significantly more efficient or numerically robust evaluation strategies, or improved interoperability through the explicit representation of a widely recognized abstraction.

3.2.3 Relationship to the HS^3 language

Primitive specifications extend the vocabulary of HS^3 but do not redefine the language itself.

Primitive specifications may introduce new functions, distributions, or regular members of any other existing top-level HS^3 category (see Section Top-level components). They shall not introduce new language constructs. In particular, a primitive shall not change its semantic role based on configuration or context, create or transform other serialized objects, or introduce control-flow, parser directives, or other metaprogramming facilities. Such proposals constitute extensions of the HS^3 language and require separate specification and review.

3.2.4 Required specification

Every primitive specification shall define:

- its mathematical semantics, or an equivalent algorithmic definition;
- the meaning of every serialized field, including the relationship between serialized objects and the mathematical quantities they represent;
- any normative default values;
- sufficient examples to remove ambiguity.

Primitive specifications SHOULD define the normative semantics of a primitive independently of any particular implementation. Whenever practical, this SHOULD be done by specifying the mathematical object represented by the primitive. Where no sufficiently precise mathematical characterization exists, or where the algorithm itself defines the abstraction, a normative algorithmic specification may be used instead.

The amount of explanatory material SHOULD be proportional to the complexity of the primitive. Simple primitives may require little more than their mathematical definition and serialization. More complex primitives SHOULD include whatever additional explanation is necessary to enable independent conforming implementations. Informative mappings to existing software are encouraged but never replace the normative definition.

3.2.5 Serialization

Every serialized field shall have a well-defined mathematical meaning.

Serialized objects SHOULD expose mathematical structure wherever practical. Mathematical concepts such as coefficients, tensors, domains, or observables SHOULD be preferred over implementation-specific concepts such as object handles or memory layouts.

Implementation-specific state may be serialized where necessary, but SHOULD remain self-contained, inspectable, schema-validatable, and editable whenever practical. References to external binary objects SHOULD be avoided unless no practical alternative exists.

3.2.6 Defaults and miscellaneous information

Evaluation shall be completely determined by the serialized object together with any normative defaults defined by the primitive.

Optional fields that affect evaluation shall have unique specification-defined default values. Such defaults shall not depend on implementations, external libraries, software versions, or backend-specific conventions.

Information that has no effect on mathematical evaluation shall be stored in `misc`. Conforming evaluators shall produce equivalent results regardless of the contents of `misc`, although implementations may use it for diagnostics, visualization, optimization, or internal representation.

3.2.7 Existing implementations

Where a primitive represents functionality already established in an existing framework or scientific domain, its semantics SHOULD remain recognizable to domain experts while preserving implementation-independent mathematical meaning.

Primitive specifications SHOULD minimize the conceptual distance between established domain abstractions and their HS^3 representation. Generalization is appropriate where it improves interoperability or removes accidental implementation details. It SHOULD be avoided where it obscures established abstractions or complicates import, export, or implementation without providing corresponding semantic benefit.

3.2.8 Alternative implementations

An enum-like field may select between alternative methods when those methods operate on the same serialized state and the meaning of all other fields remains unchanged. The selected method may alter how that state is interpreted or evaluated, but shall not determine which disjoint set of fields is applicable.

If different enum values require substantially different configuration data, parameterizations, or validity conditions, they SHOULD generally be represented as separate primitives rather than as variants of a single primitive.

Implementations need to support only a subset of the methods defined by a primitive specification. An implementation that does not support the requested method shall not silently substitute another method. It shall either reject the object or explicitly report the use of a semantically valid fallback.

3.2.9 Conformance

A primitive specification is complete if independent implementations can reproduce equivalent mathematical behaviour from the specification alone, together with the documentation of any explicitly referenced external libraries.

Normative text defines behaviour. Informative text may provide motivation, examples, implementation guidance, historical context, or mappings to existing software, but does not contribute to the formal definition of the primitive.

3.2.10 Editorial Guideline

As a practical guideline, contributors should continuously ask:

> Could an independent, faithful implementation of this primitive be written without ambiguity from this specification alone, together with the documentation of any explicitly referenced external libraries?

If the answer is no, the specification is likely missing semantic or algorithmic information.

3.3 HistFactory

HistFactory, documented by Cranmer et al. in 2012 [11], models bin counts as independent Poisson random variables whose expectations come from a mixture of histogram templates. Its random variables are the counts n_i; x is the observable (or tuple of observables) used to define the fixed bins B_i.

A *channel* is a selected event category or analysis region with its own observed histogram, such as a signal or background control region. Each prediction adds the contributions of *samples* (signal and background processes) sharing the channel's binning. Parameter variations change yields, not bin boundaries; different channels may use different binning.

The sample yields and normalized shapes define the event intensity:

$$\rho(x;\theta) = \sum_s N_s(\theta) f_s(x;\theta), \qquad \int f_s(x;\theta)\,dx = 1, \qquad \lambda_i(\theta) = \int_{B_i} \rho(x;\theta)\,dx.$$

Each sample has a yield N_s and a normalized shape f_s over the channel's observable region. The intensity integrates to the total expected yield $\sum_s N_s$,

rather than to one. HistFactory stores bin yields directly; the decomposition into yields and normalized shapes is explanatory. The parameters θ may include both parameters of interest, such as a signal strength, and nuisance parameters describing uncertainties.

The function `histfactory` evaluates the expected bin yields $\lambda_i(\theta)$. The distribution `histfactory_dist` additionally specifies the observation model

$$P(\mathbf{n} \mid \theta) = \prod_i \mathrm{Pois}(n_i \mid \lambda_i(\theta)), \qquad n_i \in \{0, 1, 2, ...\},$$

normalized over count vectors. Independence and Poisson fluctuations are modeling assumptions beyond the expected-yield prediction. Both representations use the samples, modifiers and constraint terms described below; their `type` entries differ. Figure 1 illustrates the sample contributions (originally from [12]).

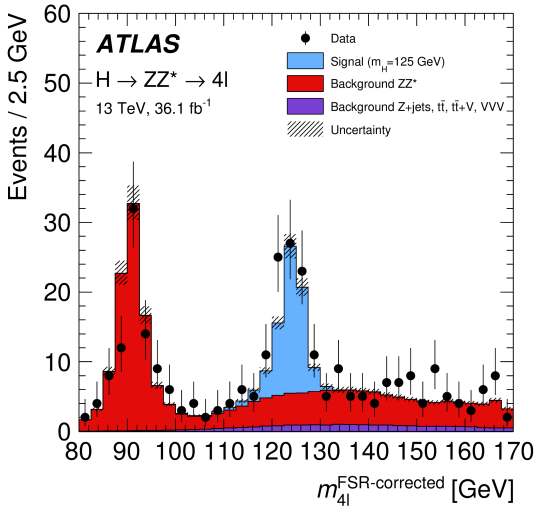


Figure 1: A binned statistical model describing a High Energy Physics measurement, in this case of the $H \to 4l$ process by the ATLAS collaboration. Three different sample (blue, red, violet) are considered.

In the modifier formula below, $\lambda(x)$ is a step function whose value in B_i is the bin yield $\lambda_i(\theta)$. For continuous bins of width or volume $|B_i|$, the corresponding piecewise-constant intensity is $\rho(x;\theta) = \lambda_i(\theta)/|B_i|$; integrating $\lambda(x)$ itself would incorrectly introduce a bin-volume factor.

$$\lambda(x) = \sum_{s \in \text{samples}} \left[\left(d_s(x) + \sum_{\delta \in M_\delta} \delta(x, \theta_\delta) \right) \prod_{\kappa \in M_\kappa} \kappa(x, \theta_\kappa) \right]$$

Here $d_s(x)$ is the prediction associated with the sample s, a step function

$$d_s(x) = \chi_b^{y_s}(x)$$

In this section, $\chi_b^{y_s}(x)$ denotes a generic step function in the binning b such that $\chi_b(x) = y_{s,i}$, some constant, if $x \in [b_i, b_{i+1})$. The $y_{s,i}$ in this case are the bin contents (yields) of the histograms. The M_κ are the multiplicative modifiers, the M_δ are the additive modifiers. Each of the modifiers is either multiplicative (κ) or additive (δ). All samples and modifiers share the same binning b. The modifiers depend on the model parameters θ. Each modifier can only depend on one θ_i, but the θ_i can take the form of vectors and the same θ_i can be shared by several modifiers. By convention, these are denoted α if they affect all bins in a correlated way, and γ if they affect only one bin at a time. The types of modifiers are

- A *uncorrelated shape systematic* or `shapefactor` modifier is a multiplicative modifier that scales each single bin by the value of some independent parameter γ. Here, $\theta_i = \vec{\gamma}$, where the length of $\vec{\gamma}$ is equal to the number of bins in this region. *This type of modifier is sometimes called `shapesys`, with some nuance in the meaning. However, both are synonymous in the context of this standard.*
- A *correlated shape systematic* or `histosys` modifier is an additive modifier that adds or subtracts a constant step function χ^f, scaled with a single factor α. The modifier contains a `data` section, which contains the subsections `hi` and `lo` that help to define the step function χ^f. They contain `contents`, which define the bin-wise additions or subtractions for $\alpha = 1$. The functional form of χ^f for intermediate and outlying α is set by the `interpolation` component, described below. Here, $\theta_i = \alpha$.
- A *normalization systematic* or `normsys` modifier is a multiplicative modifier that scales the entire sample with the same constant factor f that is a function of α. The modifier contains a `data` section, which contains the values `hi` and `lo` that help to define f. There are different functional forms that can be chosen for f. However, by convention $f(\alpha = 0) = 1$, $f(\alpha = +1) =$"`hi`" and $f(\alpha = -1) =$"`lo`". The functional form of f for intermediate and outlying α is set by the `interpolation` component, described below. In this case, $\theta_i = \alpha$.
- A *normalization factor* or `normfactor` modifier is a multiplicative modifier that scales the entire sample in this region with the value of the parameter μ itself. In this case, $\theta_i = \mu$.
- The `staterror` modifier is a shorthand for encoding uncorrelated statistical uncertainties on the values of the step-functions, using a variant[1] of the Barlow-Beeston Method [13]. Here, the relative uncertainty on the sum of all samples in this region containing the `staterror` modifier is

[1]The variation consists of summarizing all contributions in the stack to a single contribution as far as treatment of the statistical uncertainties is concerned.

computed bin-by-bin. Then, a constrained *uncorrelated shape systematic* (`shapesys`) is created, encoding these relative uncertainties in the corresponding `Poisson` (or `Gaussian`) constraint term.

The different modifies and their descriptions are also summarized in the following table:

Type of Modifier	**Description**	**Definition**	**Free Parameters**	**Number of Free Parameters**
`histosys`	Correlated Shape systematic	$\delta(x,\alpha) = \alpha * \chi_b^f$	α	1
`normsys`	Normalization systematic	$\kappa(x,\alpha) = f(\alpha)$	α	1
`normfactor`	Normalization factor	$\kappa(x,\mu) = \mu$	μ	1
`shapefactor`, `staterror`	Shape factor	$\kappa(x,\vec{\gamma}) = \chi_b^\gamma$	γ_0, ..., γ_n	#bins

The `staterror` modifier is a special subtype of `shapefactor`, where the mean of the constraint is given as the sum of the predictions of all the samples carrying a `staterror` modifier in this bin.

The way modifiers affect the yield in the corresponding bin is subject to an interpolation function. The `histosys` and `normsys` modifiers thus allow for an additional component `interpolation`, a struct with the components

- `type`: how the response combines with the sample prediction, `add` or `mult`, for additive or multiplicative, respectively,
- `in`: the interpolating function used inside the bounds, $|\theta| < 1$,
- `out`: the extrapolating function used outside the bounds, $|\theta| \geq 1$, if `null` the same function as for interpolation is used.

The final modified prediction $y(\theta_1, \ldots, \theta_n)$ is obtained by combining the sample's nominal prediction y_0 with a normalized response function $I(\theta; y_-, y_0, y_+)$ for each parameter θ_i. Unlike y, I is anchored relative to y_0 rather than to it directly, with the anchor values depending on the combination `type`:

- For `type="add"`: $I(0) = 0$, $I(+1) = y_+ - y_0$, $I(-1) = y_- - y_0$
- For `type="mult"`: $I(0) = 1$, $I(+1) = y_+/y_0$, $I(-1) = y_-/y_0$

For `type="mult"`, the combined prediction follows as:

$$y(\theta_1, \ldots, \theta_n) = y_0 \cdot \prod_i I(\theta_i; y_{i,-}, y_0, y_{i,+}),$$

while for `type="add"` the combination follows as:

$$y(\theta_1, \ldots, \theta_n) = y_0 + \sum_i I(\theta_i; y_{i,-}, y_0, y_{i,+}).$$

Here $y_{i,-}$ and $y_{i,+}$ denote the `lo`/`hi` variations associated with parameter θ_i; y_0, the sample's nominal prediction, is shared across all parameters and enters only through the combination above, never through I itself.

The following functions are allowed for `in` and `out`:

- `poly1`: linear function
- `poly2`: parabolic function
- `poly6`: degree-6 polynomial
- `exp`: exponential function

The choice of functions for `in` and `out`, together with the anchor conditions above and, where required, matching of first and second derivatives at the boundary ($\frac{\mathrm{d}I}{\mathrm{d}\theta}\big|_{\theta=\pm1}$ and $\frac{\mathrm{d}^2 I}{\mathrm{d}\theta}\big|_{\theta=\pm1}$), fixes all free parameters of the chosen functions except for θ itself.

The following choices of I are often used and showcase the resulting functional forms for common combinations of `type`, `in` and `out`. Note that for `type="mult"`, all interior and extrapolation terms are expressed as ratios $y_\pm/y_0$ rather than raw $y_\pm$, so that the anchor condition $I(0) = 1$ holds independent of y_0; for `type="add"`, terms are expressed as differences $y_\pm - y_0$, consistent with the anchor $I(0) = 0$.

- additive, piecewise linear:
  - `{"type":"add", "in":"poly1", "out":null}`
  - with $I_1(\theta; y_-, y_0, y_+) = \begin{cases} \theta \cdot (y_+ - y_0) & \theta \geq 0 \\ \theta \cdot (y_0 - y_-) & \theta < 0 \end{cases}$
- multiplicative, piecewise exponential:
  - `{"type":"mult", "in":"exp", "out":null}`
  - with $I_2(\theta; y_-, y_0, y_+) = \begin{cases} (y_+/y_0)^\theta & \theta \geq 0 \\ (y_-/y_0)^{-\theta} & \theta < 0 \end{cases}$
- additive, quadratic interpolation and linear extrapolation
  - `{"type":"add", "in":"poly2", "out":"poly1"}`
  - with $I_3(\theta; y_-, y_0, y_+) = \begin{cases} (2s + d) \cdot (\theta - 1) + y_+ - y_0 & \theta > 1 \\ s \cdot \theta^2 + d \cdot \theta & |\theta| \leq 1 \\ -(2s - d) \cdot (\theta + 1) + y_- - y_0 & \theta < -1 \end{cases}$,
    where $s = \frac{1}{2}(y_+ + y_-) - y_0$ and $d = \frac{1}{2}(y_+ - y_-)$
- additive, polynomial (6th degree) interpolation and linear extrapolation
  - `{"type":"add", "in":"poly6", "out":"poly1"}`
  - with $I_4(\theta; y_-, y_0, y_+) = \begin{cases} \theta \cdot (y_+ - y_0) & \theta \geq 1 \\ \theta \cdot (S + \theta \cdot A(15 + \theta^2 \cdot (3\theta^2 - 10))) & |\theta| < 1 \\ \theta \cdot (y_0 - y_-) & \theta \leq -1 \end{cases}$,
    where $S = \frac{1}{2}(y_+ - y_-)$ and $A = \frac{1}{16}(y_+ + y_- - 2y_0)$

- multiplicative, polynomial (6th degree) interpolation and exponential extrapolation
  - `{"type":"mult", "in":"poly6", "out":"exp"}`
  - with $I_5(\theta; y_-, y_0, y_+) = \begin{cases} (y_+/y_0)^\theta & \theta \geq 1 \\ 1 + \theta \cdot (a + \theta \cdot (b + \theta \cdot (c + \theta \cdot (d + \theta \cdot (e + \theta \cdot f))))) & |\theta| < 1 \\ (y_-/y_0)^{-\theta} & \theta \leq -1 \end{cases}$,

    where: $\begin{array}{l} a = \frac{1}{8}(15A_0 - 7S_1 + A_2) \\ b = \frac{1}{8}(-24 + 24S_0 - 9A_1 + S_2) \\ c = \frac{1}{4}(-5A_0 + 5S_1 - A_2) \\ d = \frac{1}{4}(12 - 12S_0 + 7A_1 - S_2) \\ e = \frac{1}{8}(3A_0 - 3S_1 + A_2) \\ f = \frac{1}{8}(-8 + 8S_0 - 5A_1 + S_2) \end{array}$, with $u = y_+/y_0$, $v = y_-/y_0$,

    $\log t := \begin{cases} \log t & t > 0 \\ 0 & t \leq 0 \end{cases}$, and $\begin{array}{ll} S_0 = \frac{1}{2}(u + v) & A_0 = \frac{1}{2}(u - v) \\ S_1 = \frac{1}{2}(u \log u - v \log v) & A_1 = \frac{1}{2}(u \log u + v \log v) \\ S_2 = \frac{1}{2}(u \log^2 u + v \log^2 v) & A_2 = \frac{1}{2}(u \log^2 u - v \log^2 v) \end{array}$
- multiplicative, polynomial (6th degree) interpolation and linear extrapolation
  - `{"type":"mult", "in":"poly6", "out":"poly1"}`
  - with $I_6(\theta; y_-, y_0, y_+) = 1 + I_4(\theta; y_-/y_0, 1, y_+/y_0)$

Modifiers can be constrained. This is indicated by the component `constraint`, which identifies the type of the constraint term. In essence, the likelihood picks up a penalty term for changing the corresponding parameter too far away from its nominal value. The nominal value is, by convention, defined by the type of constraint, and is 0 for all modifiers of type `sys` (`histosys`, `normsys`) and is 1 for all modifiers of type `factor` (`normfactor`, `shapefactor`). The strength of the constraint is always such that the standard deviation of constraint distribution is 1.

The supported constraint distributions, also called constraint types, are `Gauss` for a gaussian with unit width (a gaussian distribution with a variance of 1), `Poisson` for a unit Poissonian (e.g. a continous Poissonian with a central value 1), or `LogNormal` for a unit LogNormal,. If a constraint is given, a corresponding distribution will be considered in addition to the `aux_likelihood` section of the likelihood, constraining the parameter to its nominal value.

An exception to this is provided by the `staterror` modifier as described above, and the `shapesys` for which a Poissonian constraint is defined with the central values defined as the squares of the values defined in `vals`.

The constraint terms belong to the prediction and not to its representation: a likelihood that contains a `histfactory_dist`, or a distribution that depends on a `histfactory` function, considers the constraint terms of all modifiers of that prediction in addition to the `aux_likelihood` section of the likelihood, in the same way for both representations.

The components of a HistFactory prediction are:

- `name`: custom unique string
- `type`: `histfactory` (function) or `histfactory_dist` (distribution)
- `axes`: array of structs representing the axes. If given each struct needs to have the component `name`. Further, (OPTIONAL) components are `max`, `min` and `nbins`, or, alternatively, `edges`. The definition of the axes follows the format for binned data (see Section Binned Data).
- `default_interpolation`: OPTIONAL struct defining the default interpolation behaviour, for modifiers that do not specify it on their own. It has the components `type`, `in` and `out`, as described above.
- `samples`: array of structs containing the samples of this channel. For details see below. Struct of one sample:
  - `name`: (OPTIONAL) custom string, unique within this function
  - `data`: struct containing the components `contents` and `errors`, depicting the data contents and their errors. Both components are arrays of the same length.
  - `modifiers`: array of structs with each struct containing the modifiers for this sample. Struct of one modifier:
    * `type`: type of the modifier
    * `parameter` or `parameters`: defining a string or an array of strings, respectively. Relating to the name or names of parameters controlling this modifier. Two modifiers are correlated exactly if they share the same parameters as indicated by `parameter` or `parameters`. In such a case, it is mandatory that they share the same constraint term. If this is not the case, the behavior is undefined.
    * `data`: OPTIONAL relevant data for modifier. Its format depends on `type`, which is described above.
    * `constraint`: OPTIONAL definition of how the modifier is constrained. Its format depends on `type`, which is described above.
    * `interpolation`: OPTIONAL struct defining how the effect of the modifier is calculated within and outside of the boundaries $\theta = \pm 1$. It has the components `type`, `in` and `out` as described above. If this is omitted here for at least one modifier the top-level component `default_interpolation` becomes REQUIRED.

HistFactory

```
{
  "name": "myAnalysisChannel",
  "type": "histfactory_dist",
  "axes": [
        { "max": 1.0, "min": 0.0, "name": "myRegion", "nbins": 2 }
  ],
  "name":"myChannel1",
  "default_interpolation": { "type": "mult", "in": "poly6", "out": "exp" },
  "samples": [
    {
      "name": "mySignal",
```

```
      "data": { "contents": [ 0.5, 0.7 ], "errors": [ 0.1, 0.1 ] },
      "modifiers": [
        { "parameter": "Lumi", "type": "normfactor" },
        { "parameter": "mu_signal_strength", "type": "normfactor" },
        { "constraint": "Gauss", "data": { "hi": 1.1, "lo": 0.9 },
          "parameter": "my_normalization_systematic_1",
          "type": "normsys" },
        { "constraint": "Poisson", "type": "staterror",
          "parameters": ["gamma_stat_1","gamma_stat_2"]},
        { "constraint": "Gauss", "type": "histosys",
          "data": {
            "hi": { "contents": [ -2.5, -3.1 ] },
            "lo": { "contents": [ 2.2, 3.7 ] }
           },
           "interpolation": { "type": "add", "in": "poly6", "out": "poly1" },
           "parameter": "my_correlated_shape_systematic_1" },
        { "constraint": "Poisson", "data": { "vals": [ 0.0, 1.2 ] },
          "parameter": "my_uncorrelated_shape_systematic_2",
          "type": "shapesys" }
      ]
    },
    {
      "name": "myBackground",
      ...
   }
  ]
}
```

References

[1] Creative Commons. *CC0 license*. URL: https://creativecommons.org/publicdomain/zero/1.0.

[2] Kyle Cranmer et al. "Publishing statistical models: Getting the most out of particle physics experiments". In: *SciPost Physics* 12.1 (Jan. 2022). ISSN: 2542-4653. DOI: 10.21468/scipostphys.12.1.037. URL: http://dx.doi.org/10.21468/SciPostPhys.12.1.037.

[3] Lukas Heinrich, Matthew Feickert, and Giordon Stark. *pyhf*. Version 0.7.0. DOI: 10.5281/zenodo.1169739. URL: https://github.com/scikit-hep/pyhf/releases/tag/v0.7.0.

[4] Lukas Heinrich et al. "pyhf: pure-Python implementation of HistFactory statistical models". In: *Journal of Open Source Software* 6.58 (2021), p. 2823. DOI: 10.21105/joss.02823. URL: https://doi.org/10.21105/joss.02823.

[5] The ROOT Team. *root-project/root*. Version 6.26. Mar. 2022.

[6] Scott O. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Mar. 1997. DOI: 10.17487/RFC2119. URL: https://www.rfc-editor.org/info/rfc2119.

[7] *The JSON data interchange syntax*. ISO/IEC 21778. Nov. 2017. URL: https://www.iso.org/standard/71616.html.

[8] Roger Barlow. "Extended maximum likelihood". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 297.3 (1990), pp. 496–506. ISSN: 0168-9002. DOI: https://doi.org/10.1016/0168-9002(90)91334-8. URL: https://www.sciencedirect.com/science/article/pii/0168900290913348.

[9] Glen Cowan et al. "Asymptotic formulae for likelihood-based tests of new physics". In: *Eur. Phys. J. C* 71 (2011). [Erratum: Eur.Phys.J.C 73, 2501 (2013)], p. 1554. DOI: 10.1140/epjc/s10052-011-1554-0. arXiv: 1007.1727 [physics.data-an].

[10] R. L. Workman et al. "Review of Particle Physics". In: *PTEP* 2022 (2022), p. 083C01. DOI: 10.1093/ptep/ptac097.

[11] Kyle Cranmer et al. *HistFactory: A tool for creating statistical models for use with RooFit and RooStats*. Tech. rep. New York: New York U., Jan. 2012. DOI: 10.17181/CERN-OPEN-2012-016. URL: https://cds.cern.ch/record/1456844.

[12] ATLAS Collaboration. "Measurement of inclusive and differential cross sections in the $H \to ZZ^* \to 4\ell$ decay channel in pp collisions at $\sqrt{s} = 13$ TeV with the ATLAS detector". In: *JHEP* 10 (2017), p. 132. DOI: 10.1007/JHEP10(2017)132. arXiv: 1708.02810 [hep-ex].

[13] Roger Barlow and Christine Beeston. "Fitting using finite Monte Carlo samples". In: *Computer Physics Communications* 77.2 (1993), pp. 219–228. ISSN: 0010-4655. DOI: https://doi.org/10.1016/0010-4655(93)90005-W. URL: https://www.sciencedirect.com/science/article/pii/001046559390005W.